

AI Assignment #1

FasterNet for Robust Digit Pattern Recognition
under Geometric and Noise Corruptions

CHE CHI HIN, Angus
Student ID: UC325182
CISC3024 Pattern Recognition

Abstract

Thesis: a recent efficient Deep CNN (FasterNet with Partial Convolution) plus robust augmentation and rotation TTA recognizes digits under geometric/noise corruptions far better than clean-only training, and remains competitive with a student-scale equivariant baseline while staying easy for an AI agent to implement and verify. This report follows the required structure: (1) how AI was asked to find the algorithm, (2) algorithm description, (3) how AI implemented it, (4) experiment settings and results, (5) what was learnt, and (6) the source-code link. We evaluate MNIST, EMNIST Digits and SVHN. Search, coding, experiments and drafting were done by a Cursor AI agent under my direction (no hand-written code).

1 How I Asked AI Tools to Find the Algorithm

I used **Cursor** as the only AI tool for search, coding and writing; I did not hand-write code. Concretely, I first asked the agent to *plan and ask clarifying questions* rather than jump into a week-long schedule, because the Moodle brief was short. I then locked constraints in plain language:

- Find a **recent** (2023–2024) Deep CNN/Autoencoder with a peer-reviewed paper; do not copy a peer’s **RepViT** + 8×8 fusion story.
- Prefer an operator we can **verify** (not accuracy-only), and frame the application as **robust digit recognition** (a rotated “3” should still be recognized).
- After early demos looked “only digit 3 / only 90° ”, I told the agent to enrich multi-digit, multi-angle evidence and to extend to EMNIST/SVHN.

The agent proposed several lightweight CNNs; I chose **FasterNet** (Chen et al., CVPR 2023) for Partial Convolution (PConv), which is distinct from re-parameterization. Diary: `process_log.md`.

2 Algorithm Description

FasterNet / PConv.. FasterNet argues that reducing FLOPs alone (e.g., depthwise convolution) may not maximize latency because of memory access cost [1]. **Partial Convolution** applies a spatial 3×3 convolution to only a fraction $r=1/n_{\text{div}}$ of channels (paper default $r=1/4$) and leaves the remaining channels untouched; a residual pointwise MLP then mixes channels.

Our adaptation (FasterNet-mini).. For 28×28 grayscale (MNIST/EMNIST) and 32×32 RGB (SVHN) we use a 2×2 patch stem and three stages (embed 32, depths (1, 2, 2)). Each block is $\text{PConv}(n_{\text{div}}) \rightarrow \text{BN} + \text{activation} + \text{pointwise MLP}$. Main runs use ReLU; we ablate GELU. We also implement a student-scale **p4 orbit-convolution** baseline for literature comparison (augmentation vs architectural invariance) [7].

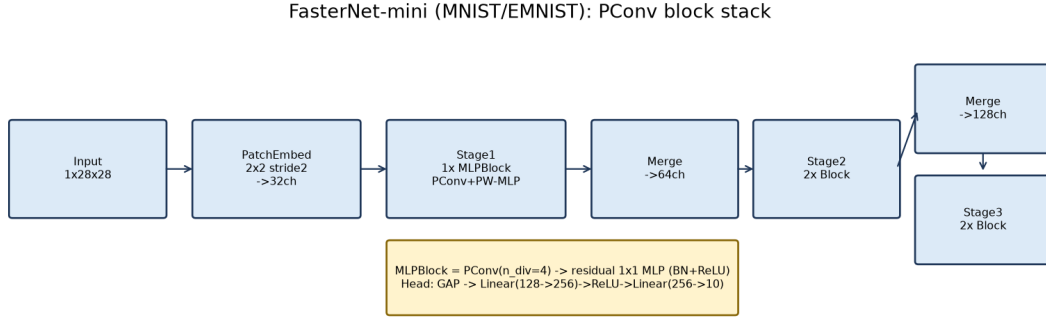


Figure 1: FasterNet-mini architecture used for MNIST/EMNIST.

PConv verification.. Untouched channels remain exact identities (max abs error = 0). At $n_{div}=4$, PConv uses 6.25% of dense 3×3 FLOPs on a fixed activation shape.

Related context.. MobileOne [2] targets mobile latency via re-parameterization (different mechanism). Corruption-style evaluation follows Hendrycks & Dietterich [3]. Digit domains: MNIST [4], EMNIST Digits [5], SVHN [6].

3 How AI Implemented the Algorithm

Under my direction, the Cursor agent implemented the full pipeline with **no hand-written student code**:

- `models/fasternet_mini.py` — PConv, MLPBlock, FasterNet-mini / T0-lite, GELU option.
- `models/equivariant_cnn.py` — p4 orbit-CNN baseline; `models/baseline_cnn.py` — SimpleCNN control.
- `corruptions.py` — train-time robust aug + test corruptions (rotate/noise/blur/shift/occlusion/elastic).
- `train.py` / `evaluate.py` — MNIST/EMNIST/SVHN loaders, MPS training, Rot90 TTA, reject curves, figures JSON.
- Analysis: `scripts/pconv_efficiency.py`, `scripts/analyze_extra.py`, `scripts/make_rich_figures.py`.
- Drivers: `run_all.py`, `scripts/run_upgrade_15.py`, `scripts/run_upgrade_gelu_eq_svhn.py`.
- Report/demo: `report.tex`, `scripts/app_gradio.py`, `process_log.md`.

I set goals, approved FasterNet, requested EMNIST/SVHN, multi-seed stats, GELU/equivariant/longer SVHN, and PDF polish; the agent wrote, ran and iterated the code.

4 Experiment Settings and Results

Settings. AdamW (10^{-3}), cosine schedule, batch 128, Apple MPS; main MNIST/EMNIST runs use 12 epochs (SVHN ablation: 12 vs 24). Robust-aug includes cardinal/continuous rotations and noise/blur/shift/occlusion/elastic. EMNIST/SVHN use 60k/10k student-scale subsamples. Metrics: corruption accuracy, Rot90+TTA, reject@0.7, ECE, multi-seed mean $\pm$ std; ablations on n_{div} , GELU and equivariant baseline.

Table 1: Main robustness results.

Setting	Clean	Rot45	Rot90	Rot90+TTA	Noise
MNIST FasterNet-aug	0.987	0.966	0.887	0.948	0.891
EMNIST FasterNet-aug	0.988	0.968	0.835	0.941	0.801
MNIST clean-only	0.993	0.588	0.127	0.659	0.377
MNIST Baseline-aug	0.958	0.735	0.267	—	0.934
SVHN 12ep	0.789	0.576	0.572	0.720	0.641
SVHN 24ep long	0.858	0.711	0.671	0.792	0.693
GELU (MNIST)	0.988	0.973	0.866	0.935	0.857
p4 Equivariant-aug	0.964	0.912	0.823	0.927	0.831

Multi-seed MNIST Rot90: $84.1\% \pm 10.0\%$. EMNIST multi-seed Rot90: $81.2\% \pm 3.2\%$. Clean ECE ≈ 0.0071 ; hardest Rot90 pair $6 \leftrightarrow 9$. Full $n_{\text{div}} \in \{2, 4, 8\}$ 12-epoch ablation (Table 2) supports paper default $r=1/4$ for latency/clean trade-off.

Table 2: n_{div} ablation (12-epoch robust-aug MNIST).

n_{div}	Params	Clean	Rot45	Rot90	Noise	Lat. (ms/512)
2	342k	0.988	0.967	0.834	0.935	14.5
4	271k	0.987	0.966	0.887	0.885	12.1
8	253k	0.977	0.965	0.930	0.838	15.5

Table 3: Clean-only training (no rotation aug): FasterNet vs p4 equivariant. Equivariant is slightly better at Rot90/TTA but both remain weak without augmentation—so aug+TTA stays essential.

Model (clean train)	Clean	Rot45	Rot90	Rot90+TTA
FasterNet-clean	0.993	0.588	0.127	0.659
p4 Equivariant-clean	0.989	0.436	0.174	0.698

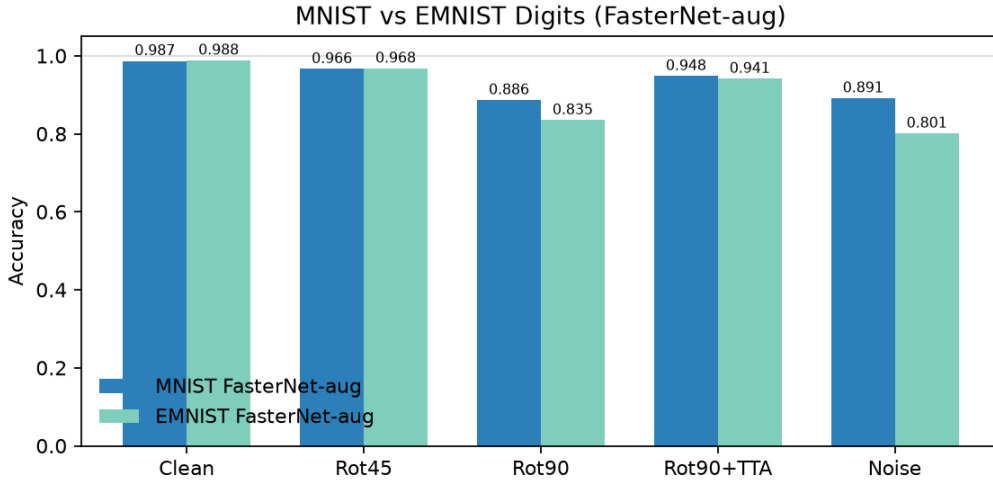
**Figure 2:** MNIST vs EMNIST Digits robustness.



Figure 3: MNIST digits 0–9 under multiple rotation angles.

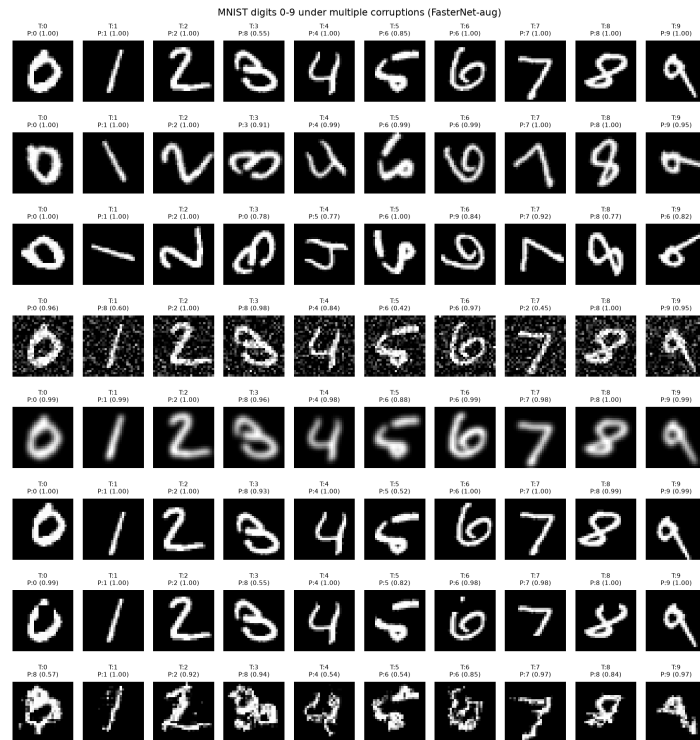


Figure 4: MNIST digits 0–9 under multiple corruptions (noise, blur, shift, occlusion, elastic)—not only rotation.

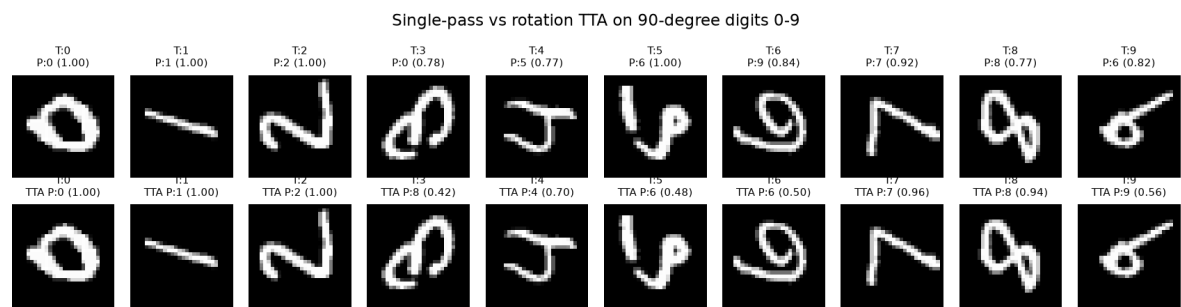


Figure 5: Single-pass Rot90 vs rotation TTA on digits 0–9; TTA recovers many orientation mistakes.

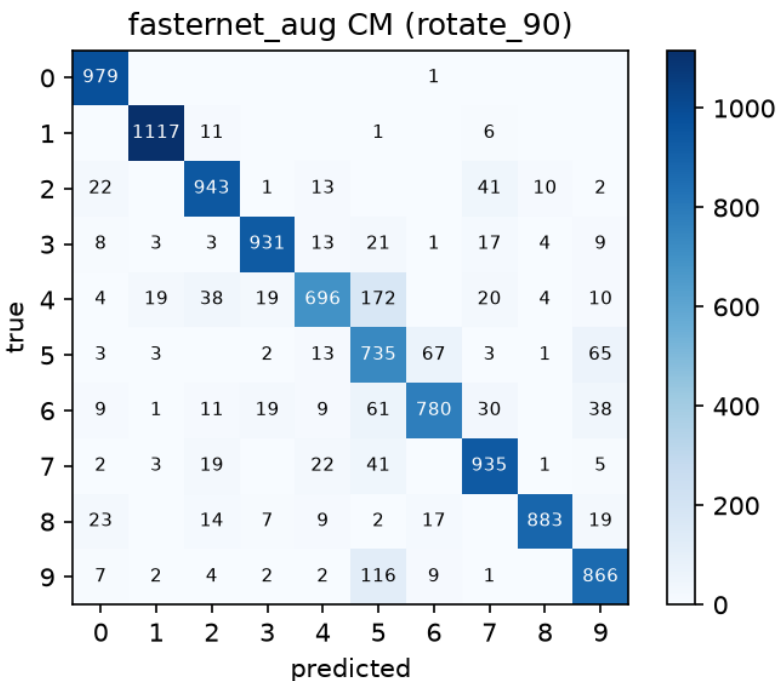


Figure 6: MNIST confusion matrix at 90° rotation; errors concentrate on confusable pairs such as 6 ↔ 9.

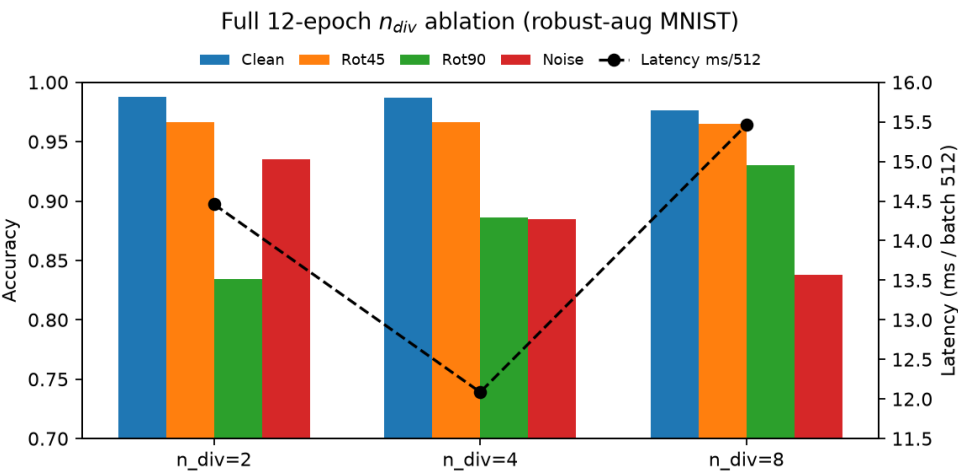


Figure 7: n_{div} accuracy–latency trade-off.

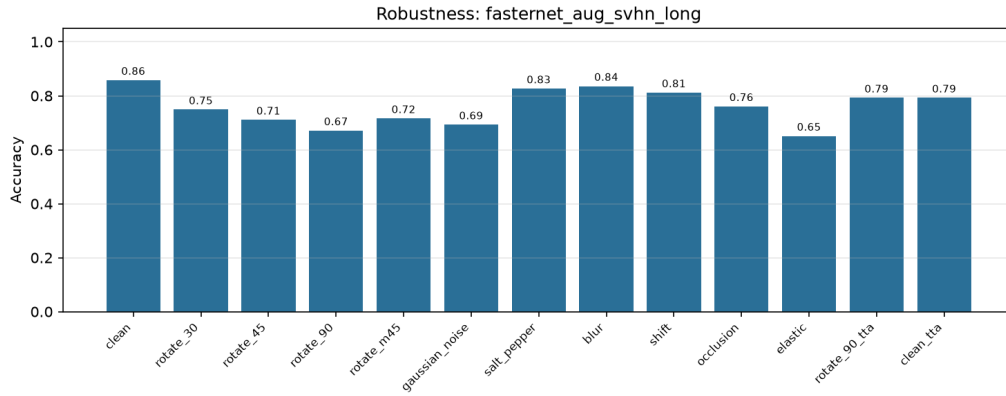


Figure 8: SVHN corruption bars after 24-epoch training.

5 What I Have Learnt from This AI Assignment

- **Prompting is the real coursework:** when I only said “train a CNN,” results looked like toy upright MNIST. When I insisted on PConv checks, multi-seed bars, EMNIST/SVHN and “not only digit 3,” the project became a pattern-recognition study.
- **Clean accuracy lies:** clean-only FasterNet is $\sim 99\%$ upright but $\sim 13\%$ at Rot90; robust aug + TTA recovers $\sim 80\text{--}94\%$. That gap is the lesson, not the headline clean score.
- **Architecture vs augmentation:** training a p4 equivariant CNN *without* rotation aug tests whether invariance can come from the model itself (Table 3). FasterNet still needs aug for strong Rot90, so I keep FasterNet+aug+TTA as the main story and equivariance as a calibrated baseline.
- **Mechanism + variance matter:** PConv identity/FLOPs, n_{div} ablation and $\text{mean} \pm \text{std}$ stop an AI report from looking like a single lucky run.
- **Domains matter:** EMNIST writer style and SVHN house numbers punish “MNIST-only” narratives; longer SVHN training fixed a large under-training gap.
- **Process logs help:** `process_log.md` is required and also made debugging AI iterations easier.

6 Source Code Webpage Link

Source code:

<https://github.com/AngusJai/CISC3024-AI-Assignment1>

Includes `models/`, training/evaluation scripts, `scripts/`, `process_log.md`, and `report.tex`. Datasets (`data/`) and `.venv/` are gitignored. A trained MNIST FasterNet-aug checkpoint is attached as a GitHub Release asset for demo convenience.

7 Conclusion

FasterNet with Partial Convolution, directed end-to-end through Cursor, is a credible recent Deep CNN for robust digit pattern recognition: verified PConv behaviour, strong multi-condition results on MNIST/EMNIST/SVHN versus clean-only collapse, ablations (including clean equivariant vs clean FasterNet), and a public source link. The assignment taught me that supervising AI—constraints, verification and honest limits—is as important as the model itself.

References

- [1] J. Chen et al. Run, Don’t Walk: Chasing Higher FLOPS for Faster Neural Networks. CVPR, 2023.

- [2] P. K. A. Vasu et al. MobileOne: An Improved One Millisecond Mobile Backbone. CVPR, 2023.
- [3] D. Hendrycks and T. Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. ICLR, 2019.
- [4] Y. LeCun et al. Gradient-based learning applied to document recognition. Proc. IEEE, 1998.
- [5] G. Cohen et al. EMNIST: an extension of MNIST to handwritten letters. 2017.
- [6] Y. Netzer et al. Reading Digits in Natural Images with Unsupervised Feature Learning. NIPS Workshop, 2011.
- [7] M. Rath and A. Condurache. Deep Neural Networks with Efficient Guaranteed Invariances. AIS-TATS, 2023.